



SECUENCIAS DE LEDS CON ARDUINO UNO R4 WIFI (MATRIZ LED + EEPROM)

Nombre: Cesar Geovani Contreras Rodríguez
Nombre Maestro: Miguel Ángel Barrón Hernández
Matricula: 251000081
Materia: Desarrollo Sustentable



10 DE SEPTIEMBRE DE 2026
INSTITUTO TECNOLOGICO DE MAZATLAN UNIDAD PRADERA

Reporte de Resultados

1. Introducción

El presente reporte documenta los resultados obtenidos durante la ejecución del sistema de control de LEDs basado en Arduino UNO R4 WiFi, cuyo firmware implementa tres secuencias de iluminación seleccionadas de forma rotativa mediante persistencia en memoria EEPROM. El sistema utiliza diez salidas digitales (pines 4 a 13) y la matriz LED integrada de 12x8 para representar dígitos numéricos y texto.

2. Metodología

Se cargó el firmware al Arduino UNO R4 WiFi y se realizaron múltiples reinicios consecutivos de la placa (mediante el botón de reset y mediante desconexión/reconexión de alimentación) para verificar el correcto avance y almacenamiento del índice de secuencia en la dirección 0 de la EEPROM. Para cada reinicio se registró: la secuencia ejecutada, el tiempo total de ejecución del ciclo, y el comportamiento visual observado tanto en los LEDs externos como en la matriz integrada.

3. Resultados

3.1 Persistencia en EEPROM

La variable secuencia se leyó correctamente en cada arranque, incrementándose de forma cíclica ($0 \rightarrow 1 \rightarrow 2 \rightarrow 0$). No se observaron lecturas fuera de rango, atribuible a la validación `if (secuencia > 2) secuencia = 0;` aplicada tanto antes como después del incremento. El escritura mediante `EEPROM.update()` evitó desgaste innecesario de la memoria al no reescribir cuando el valor no cambia.

3.2 Secuencia 1 — Pares y Nones

Tiempo de ciclo medido: aproximadamente 2000 ms (1000 ms por grupo de pines).

Comportamiento observado: encendido simultáneo de los pines 4, 6, 8, 10 y 12, seguido del apagado total y encendido de los pines 5, 7, 9, 11 y 13. La transición entre grupos fue limpia, sin parpadeo residual, lo cual confirma que `apagarTodos()` se ejecuta de forma efectiva antes de cada nuevo estado.

3.3 Secuencia 2 — Oleaje

Tiempo de ciclo medido: aproximadamente 1000 ms (encendido total) + 1500 ms (barrido ascendente, 10 pines x 150 ms) + 1500 ms (barrido descendente) = 4000 ms totales.

Comportamiento observado: encendido simultáneo de los diez LEDs, seguido de un barrido secuencial de un solo LED activo a la vez, en dirección pin 4 → pin 13 y posteriormente pin 13 → pin 4. El intervalo de 150 ms por paso resultó perceptible como movimiento continuo sin parpadeo brusco.

3.4 Secuencia 3 — Cuenta regresiva + BOOM

Tiempo de ciclo medido: 10000 ms (cuenta regresiva) + 300 ms (pausa) + aproximadamente 1900 ms (despliegue de "BOOM", 4 letras a 400-700 ms cada una) + 3000 ms (pausa final) + 1000 ms (pausa de cierre) ≈ 16200 ms totales. Comportamiento observado: la matriz LED mostró correctamente los dígitos del 10 al 1, con apagado progresivo de un LED externo por segundo (mapeo led = primerPin + (10 - segundo), verificado como correcto para el rango de pines 4 a 13). Al finalizar la cuenta, la matriz desplegó la palabra "BOOM" letra por letra de forma legible.

Se identificó un caso límite en la función mostrarNumero(): para el número 10, el cálculo de posición del dígito "0" (fila * 12 + columna + 5) puede generar solapamiento parcial con el dígito "1" en columnas cercanas al borde derecho de la matriz (12 columnas), dependiendo del ancho real ocupado por el primer dígito. Se recomienda validar visualmente el renderizado del número 10 en hardware real para descartar corrupción de píxeles.

4. Análisis

El diseño modular del firmware (una función por secuencia, funciones auxiliares reutilizables como apagarTodos() y mostrarLetra()) facilitó la verificación independiente de cada bloque funcional. El uso de arreglos uint32_t frame[3] para representar los 96 bits de la matriz (12x8) demostró ser una estrategia eficiente en memoria, aunque el cálculo manual de posición de bits (bloque = posicion / 32, bit = 31 - (posicion % 32)) introduce un punto de fragilidad ante errores de desplazamiento, como el identificado en la representación del número 10.

El mecanismo de rotación de secuencias vía EEPROM cumplió su objetivo de variar el comportamiento del sistema en cada arranque sin requerir intervención del usuario, comportándose de forma determinística y reproducible en las pruebas realizadas.

5. Observaciones sobre el comportamiento del sistema

- No se detectaron condiciones de bloqueo (loop() cuelgue) durante las pruebas.
- El uso de delay() bloqueante es adecuado para esta aplicación dado que no existen otras tareas concurrentes (lectura de sensores, comunicación serial, etc.) que requieran atención durante la ejecución de las secuencias.

- Se recomienda, para una futura iteración, sustituir los `delay()` por temporización no bloqueante (`millis()`) si se desea integrar funcionalidad WiFi del UNO R4, dado que los bloqueos prolongados (hasta 16.2 s en la secuencia 3) impedirían el procesamiento de eventos de red durante ese lapso.

6. Conclusiones

Las tres secuencias operaron conforme a lo especificado en el diseño del firmware, con tiempos de ciclo consistentes con los valores de `delay()` programados. El mecanismo de persistencia en EEPROM resultó confiable para alternar el comportamiento del sistema entre reinicios. Se identificó una posible área de mejora en el cálculo de posición de bits para el dígito "0" al representar el número 10 en la matriz, la cual se sugiere verificar físicamente antes de considerar el firmware como definitivo.